Introduction

The project consists of creating and integrating DevOps tools into the code base for https://github.com/bartoszjarocki/cv. The program is a lightweight resume template made to be hosted on the web. The repository didn't contain any DevOps solutions from the start so our goal was to get a full solution including CI/CD, testing, IaC and dependency management up and running for it. In this short report we will go over all the tools used, how they interact and what function they serve. As well as which design decisions were made and what limitations there are to the pipeline we ended up with.

1. Architecture and Processes

The setup is divided into four parts: a continuous integration pipeline, continuous deployment, infrastructure as code and dependency management. We chose to host our code in a repository on Github.

The continuous integration pipeline runs on every push and pull request to the main branch and contains two jobs that run in parallel. The first job installs dependencies, runs Biome, runs tests and builds the app. The second job checks the Terraform files with `fmt` and `validate`, so mistakes in the infrastructure code can be caught. Running them in parallel means that the pipeline runs faster.

Since the project did not have any unit tests before, we chose Jest as the testing framework. This required a short configuration file to get the tests to work with React components, and all these tests are run in the CI pipeline.

Deployment is handled by Vercel which is a platform that allows hosting and deployment of applications on the cloud. Every pull request gets its own preview link and merges to main go to production. This gives us two stages of checking: CI confirms the code works and previews let people confirm that it looks right.

Since the amount of tools for this project is relatively small, the terraform configuration has only one resource, and uses a local state file. The resource is for Vercel, which is mainly configured in the main terraform file.

Dependabot checks our npm packages once a day and opens up to five pull requests at a time. Since every Dependabot PR runs through the same CI pipeline, upgrades get tested before anyone merges them.

2. Design Decisions

For CI we chose Github actions since it's built into Github. This allows our pipeline to run automatically on every push or pull request against main. It also tests the code as it would look

after merging, not just on the feature branch. For deployment we chose Vercel because the app is built with Next.js and Vercel supports it out of the box. Terraform was added to describe the Vercel project in code, so the setup is visible in the repo instead of only existing as settings in a dashboard. Dependabot was chosen because it's Github's built-in tool for keeping dependencies up to date, so it needs no extra service or setup.

We also kept out changes narrow on purpose. The project already contained some tools for building, linting and formatting the code, so we left those alone and tried to add what was missing, CI, CD, IaC and Dependabot, on top of that so as to not have to change things too much. Terraform follows the same idea: it only describes the Vercal and its link to the Github repo, not every setting. Keeping things small makes the setup easier to read, easier to review and less likely to break.

3. How different components interact

Since the project uses pnpm, there are several different scripts defined that can be run. We have added the 'pnpm test' script, which runs all unit tests using Jest. When opening a Pull Request or pushing to main, the CI pipeline runs the tests along with linting, formatting and build while also validating the terraform configuration, as we mentioned before. We added a branch protection on Github, preventing PRs from being merged to main unless the checks pass.

Moreover, the deployment of the project with Vercel has been configured to automatically create the preview URL for Pull Requests. Similarly, the production deployment is also automatic for merges to the main branch.

Terraform does not currently apply any changes to the infrastructure in the CI pipeline. However, this can be done locally with an API key to authenticate to the Vercel API.

Dependabot does not require the CI pipeline to be executed, but has been configured to alert when vulnerabilities are found and we have set up dependabot to automatically open a PR for alerts. We also set up dependabot to be able to suggest version updates.

4. Limitations

One limitation with Vercel is that we are using the free version. This doesn't allow multiple people on the same team, which means that only the owner of the project can trigger Vercel to deploy to production. This causes some troubles if multiple people are developing a project with owner privileges, but can easily be fixed by paying for Vercel.

Another limitation and trade-off is how we use terraform. Since it only uses a local state file, the state is not shared between different workspaces and would require setting up a remote backend to do so.

A trade-off we had to make was the tests. Since the project didn't have any unit tests at all, we had to create our own. However, our tests do not cover the entire codebase and therefore passing the tests and builds does not guarantee a fully working application.

5. AI Usage

The usage of AI during the development of the project has mostly been focused on getting information about the different tools that we use. For example, we have used LLMs to find out what is needed in the configuration of Github actions and terraform. However, the code and configurations are written by us.